# Open Source in SAF

## Table of contents


The case in brief . . . 1
What the principle says . . . 1
Why the architecture matters here . . . 2
Where the framework already says this . . . 2
What it means stream by stream . . . 2
Where to go next . . . 3


*Why the framework discusses open source as part of its architecture, and not only as a purchasing decision.*

> Who this is for
>
> Anyone considering how their office should source the tools that carry statistical method: CIOs and CTOs, architects, methodologists, procurement staff. This page gathers what the framework says about open source in its drivers, goals, principles and value streams. It takes about five minutes to read.

### The case in brief

A statistical office's methods are part of what makes its figures trustworthy. When those methods run inside software that nobody outside the vendor can inspect, the office can no longer fully explain how a published figure was produced. It also cannot easily move the method elsewhere or improve it on its own timetable.

The framework returns to this point several times, using the same phrase: open source keeps **collective control of critical statistical capabilities** within the statistical community, instead of leaving offices dependent on a vendor.

### What the principle says

> SAF-P01
>
> **Actively participate in open-source communities to maintain collective control over statistical processes.**
> Statistical organisations build statistical capabilities on shared open-source software, contribute to its communities, and release software they develop for statistical production as open source by default. Non-statistical processes use standard applications.

Note the word *participate*. Using open components is not enough on its own: an office is expected to contribute fixes and methods back to the communities it relies on. A component that every office uses and nobody maintains is a shared dependency, and gives none of them real control.

SAF-P01 is one of the six principles applied by all six production value streams. See Principles to Apply in the Business Architecture chapter.

## Why the architecture matters here

This part is specific to SAF.

The production chain is built from steady states. Each value stream is defined by the artefact it hands over and the metadata that travels with it. The tool it uses internally is not part of that definition.

This has a direct consequence for tooling. If a stream's obligations are fully expressed at its boundaries, the component inside can be adopted, replaced or shared without affecting anything downstream. A community editing engine can be introduced in VS-03, and VS-04 still receives a registered Processed Data artefact with its edit lineage, as before.

The two ideas support each other. Because streams are loosely coupled, replacing a tool is a decision for one stream and does not require a migration across the whole chain. Open components, in turn, give offices something to put in those replaceable places.

For the same reason, a component built by one office can be shared: if it respects the steady-state contract, another NSO can use it without adopting the first office's internal design.

## Where the framework already says this

Table 1: Open source across the framework

| Chapter | What it says |
|---|---|
| SAF-D1 Evolving Data Ecosystems and Partnerships | Open-source development is a key element of the modern data ecosystem: a shared foundation for innovation, transparency and cost efficiency, letting offices co-develop tools and methods with peers, academia and private partners |
| SAF-G1 Cost Efficiency and Resource Sharing | Open-source components and community-driven platforms strengthen transparency, collaboration and cost efficiency, and support modernising incrementally instead of through large reinvestments |
| SAF-G4 Support for Data Innovation | Open source lets organisations build on each other's work and strengthens the collective capacity of the statistical community; SAF supports it with patterns for safe reuse of open components |
| SAF-P01 | The principle itself, applied across all six production streams |
| VS-08 Innovation and Methodological Development | Innovate on and contribute to shared, open methods and tools rather than vendor black boxes |
| VS-09 Infrastructure and Platform Enablement | Favours open, shared platform components to retain collective control |

## What it means stream by stream

Each value-stream chapter names the tooling where this applies in its domain.

Table 2: Open tooling by value stream

| Stream | Where open tooling carries the method |
|---|---|
| VS-01 Acquisition | Collection and ingestion tooling |
| VS-02 Standardization | Classification, coding and mapping tooling, for example libraries for SDMX, the statistical data-exchange standard |
| VS-03 Editing | Editing, imputation and coding engines, such as selective-editing engines and imputation libraries |
| VS-04 Estimation | Estimation, weighting and modelling tooling, such as calibration and variance-estimation libraries |
| VS-05 Composition | Composition, indicator-construction and reporting or visualisation tooling |
| VS-06 Dissemination | Dissemination platforms, APIs and visualisation tooling |

In every row, the closer a component is to statistical *method*, the stronger the case for it being open, because that is the part an office must be able to explain and defend.

## Where to go next

The principle is listed with the others in Architecture Principles, and how the principles apply across the production chain is described in Principles to Apply. For what the steady states are and why they make components replaceable, see SAF in Brief.